Reading CSV Files, Cleaning Missing Data, Filtering and Sorting in Pandas

Introduction

Data analysis begins with loading data into a program. In Python, the **Pandas** library provides powerful tools for reading, cleaning, filtering, and sorting datasets. These operations help transform raw data into meaningful information that can be analyzed effectively.

---

1. Reading CSV Files

What is a CSV File?

A **CSV (Comma-Separated Values)** file is one of the most commonly used file formats for storing tabular data. Each row represents a record, and each column represents a field separated by commas.

Example: students.csv

Student_ID	Name	Age	Course	Marks
101	Rahul	20	AI	85
102	Priya	21	Python	92
103	Aman	19	SQL	78

Importing Pandas

```
import pandas as pd
```

Reading a CSV File

```
df = pd.read_csv("students.csv")
```

```
print(df)
```

Output

```
  Student_ID   Name  Age  Course  Marks
0        101  Rahul   20      AI     85
1        102  Priya   21  Python     92
2        103   Aman   19     SQL     78
```

Reading Selected Number of Rows

```
df = pd.read_csv("students.csv", nrows=2)
print(df)
```

Display First Five Rows

```
print(df.head())
```

Display Last Five Rows

```
print(df.tail())
```

Display Column Names

```
print(df.columns)
```

Display Dataset Information

```
print(df.info())
```

This displays:

- Number of rows
- Number of columns
- Data types
- Missing values

Display Statistical Summary

print(df.describe())

It provides:

- Count
- Mean
- Standard Deviation
- Minimum
- Maximum
- Quartiles

---

2. Cleaning Missing Data

What is Missing Data?

Missing data refers to values that are not available in the dataset.

Example:

Name	Age	Marks
Rahul	20	85
Priya	NaN	92
Aman	19	NaN

Here,

- Age of Priya is missing.
- Marks of Aman are missing.

Pandas represents missing values as **NaN (Not a Number)**.

Detect Missing Values

print(df.isnull())

Count Missing Values

print(df.isnull().sum())

Output

Name 0

Age 1

Marks 1

dtype: int64

Remove Rows with Missing Values

df = df.dropna()

print(df)

Only rows without missing values are kept.

Remove Columns with Missing Values

df = df.dropna(axis=1)

Fill Missing Values with a Constant

df["Marks"] = df["Marks"].fillna(0)

Fill Missing Age with Average

average_age = df["Age"].mean()

df["Age"] = df["Age"].fillna(average_age)

Fill Missing Values Using Forward Fill

df = df.ffill()

Example

Before

20

NaN

22

After

20

20

22

Fill Missing Values Using Backward Fill

df = df.bfill()

Example

Before

20

NaN

22

After

20

22

22

---

3. Filtering Data

What is Filtering?

Filtering means selecting only those rows that satisfy a given condition.

Display Students with Marks Greater than 80

result = df[df["Marks"] > 80]

print(result)

Display Students from AI Course

result = df[df["Course"] == "AI"]

print(result)

Display Students Aged Above 20

result = df[df["Age"] > 20]

Multiple Conditions

Students whose age is above 20 and marks are greater than 80.

result = df[(df["Age"] > 20) & (df["Marks"] > 80)]

```
print(result)
```

OR Condition

```
result = df[(df["Course"] == "AI") | (df["Course"] == "Python")]
```

Using isin()

Display students from AI or SQL course.

```
result = df[df["Course"].isin(["AI", "SQL"])]


print(result)
```

Selecting Specific Columns

```
print(df[["Name", "Marks"]])
```

---

4. Sorting Data

What is Sorting?

Sorting arranges data in ascending or descending order based on one or more columns.

Sort by Marks (Ascending)

```
df = df.sort_values(by="Marks")


print(df)
```

Sort by Marks (Descending)

```
df = df.sort_values(by="Marks", ascending=False)


print(df)
```

Sort by Name

```
df = df.sort_values(by="Name")
```

Sort by Multiple Columns

```
df = df.sort_values(
  by=["Course", "Marks"],
  ascending=[True, False]
)
```

This sorts:

- Course in ascending order
- Marks in descending order

Reset Index After Sorting

```
df = df.sort_values(by="Marks")


df = df.reset_index(drop=True)


print(df)
```

Complete Example

```
import pandas as pd
```

```
# Read CSV file
df = pd.read_csv("students.csv")

# Display first rows
print(df.head())

# Check missing values
print(df.isnull().sum())

# Fill missing marks with average
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

# Filter students with marks above 80
filtered = df[df["Marks"] > 80]

print(filtered)

# Sort by marks descending
sorted_df = filtered.sort_values(by="Marks", ascending=False)

print(sorted_df)
```

Real-World Applications

- **Education:** Analyze student attendance, grades, and performance.
- **Business:** Filter sales data to identify top-selling products.
- **Healthcare:** Clean patient records with missing information.
- **Finance:** Sort transactions by date or amount.
- **Human Resources:** Filter employees by department, salary, or experience.

Best Practices

- Always inspect your dataset using `head()`, `info()`, and `describe()` before analysis.
- Check for missing values using `isnull()` before performing calculations.
- Fill missing values only when it makes sense; otherwise, remove incomplete records.
- Use filtering to focus on relevant subsets of data.
- Reset the index after sorting or filtering if needed.
- Save cleaned data to a new CSV file instead of overwriting the original dataset.

Summary

Topic	Purpose	Common Functions
Reading CSV Files	Load data into Pandas	`read_csv()`, `head()`, `tail()`, `info()`, `describe()`
Cleaning Missing Data	Handle incomplete values	`isnull()`, `sum()`, `dropna()`, `fillna()`, `ffill()`, `bfill()`
Filtering Data	Select specific records	Boolean conditions, `isin()`
Sorting Data	Arrange data in order	`sort_values()`, `reset_index()`

Sample Program 1: Read a CSV File

```
import pandas as pd


# Read CSV File

df = pd.read_csv("students.csv")
```

```
# Display Data
print(df)
```

Output

```
  Student_ID   Name   Age  Course  Marks
0        101  Rahul   20     AI     85
1        102  Priya   21  Python    92
2        103   Aman   19    SQL     78
```

Sample Program 2: Display Dataset Information

```
import pandas as pd

df = pd.read_csv("students.csv")

print("First Five Rows")
print(df.head())

print("\nLast Five Rows")
print(df.tail())

print("\nDataset Information")
print(df.info())

print("\nColumn Names")
print(df.columns)
```

Sample Program 3: Find Missing Values

```
import pandas as pd

df = pd.read_csv("students.csv")

print("Missing Values")

print(df.isnull())

print("\nNumber of Missing Values")

print(df.isnull().sum())
```

Sample Output

```
Student_ID    0
Name          0
Age           1
Course        0
Marks         2
dtype: int64
```

Sample Program 4: Remove Missing Data

```
import pandas as pd

df = pd.read_csv("students.csv")

print("Original Dataset")
```

```
print(df)

df = df.dropna()

print("\nDataset After Removing Missing Values")

print(df)
```

Sample Program 5: Fill Missing Values

```
import pandas as pd

df = pd.read_csv("students.csv")

# Replace missing Marks with Average Marks
average_marks = df["Marks"].mean()

df["Marks"] = df["Marks"].fillna(average_marks)

print(df)
```

Sample Program 6: Filter Students Based on Marks

```
import pandas as pd

df = pd.read_csv("students.csv")

high_score = df[df["Marks"] > 80]
```

```
print(high_score)
```

Output

Rahul

Priya

Sneha

Sample Program 7: Apply Multiple Conditions

```
import pandas as pd

df = pd.read_csv("students.csv")

result = df[
  (df["Marks"] > 80) &
  (df["Age"] > 20)
]

print(result)
```

Sample Program 8: Filter Using `isin()`

```
import pandas as pd

df = pd.read_csv("students.csv")

courses = df[
  df["Course"].isin(["AI", "Python"])
]
```

```
print(courses)
```

Sample Program 9: Sort Data

```
import pandas as pd

df = pd.read_csv("students.csv")

print("Ascending Order")

print(df.sort_values(by="Marks"))

print("\nDescending Order")

print(df.sort_values(by="Marks", ascending=False))
```

Sample Program 10: Complete Data Cleaning Program

```
import pandas as pd

# Read CSV File
df = pd.read_csv("students.csv")

print("Original Dataset")

print(df)
```

```
# Check Missing Values
print("\nMissing Values")

print(df.isnull().sum())

# Fill Missing Marks with Average
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

# Fill Missing Age with Average
df["Age"] = df["Age"].fillna(df["Age"].mean())

# Display Students with Marks Greater than 80
print("\nStudents Scoring Above 80")

filtered = df[df["Marks"] > 80]

print(filtered)

# Sort by Marks
print("\nSorted Data")

sorted_df = filtered.sort_values(
  by="Marks",
  ascending=False
)

print(sorted_df)

# Save Cleaned Data
```

```
sorted_df.to_csv(
  "cleaned_students.csv",
  index=False
)

print("\nCleaned dataset saved successfully.")
```

Practice Programs

Program 11

Create a CSV file containing employee details. Read the file and display only employee names and salaries.

Program 12

Read a product dataset and display products costing more than ₹1000.

Program 13

Read a customer dataset and display customers belonging to "Delhi".

Program 14

Read a student dataset and sort students by Name in alphabetical order.

Program 15

Read a sales dataset and:

- Remove missing rows.
- Fill missing sales values with the average sales.
- Display products with sales greater than ₹50,000.

- Sort the result in descending order of sales.
- Save the cleaned data into a new CSV file named `sales_cleaned.csv`.

Mini Project: Student Data Analysis

Problem Statement

A school has stored student records in a CSV file containing the following columns:

- Student_ID
- Name
- Age
- Course
- City
- Marks

Write a Python program using **Pandas** to:

1. Read the CSV file.
2. Display the first five records.
3. Display dataset information.
4. Identify missing values.
5. Fill missing marks with the average marks.
6. Display students scoring more than 80 marks.
7. Display students enrolled in the **AI** course.
8. Sort students by marks in descending order.
9. Save the cleaned and sorted data into a new CSV file named `students_cleaned.csv`.

These sample programs progress from basic to advanced and provide students with hands-on practice in reading CSV files, cleaning missing data, filtering datasets, sorting records, and exporting cleaned data.